CHAPTER

Abstract Data Types and Modules

CHAPTER 11

In Chapter 9, we defined a data type as a set of values, along with certain operations on those values. In that discussion, we divided data types into two kinds: predefined and user-defined. Predefined types, such as `integer` and `real`, are designed to insulate the user of a language from the implementation of the data type, which is machine-dependent. These data types can be manipulated by a set of predefined operations, such as the arithmetic operations, whose implementation details are also hidden from the user. Their use is completely specified by predetermined semantics, which are either explicitly stated in the language definition or are implicitly well known (like the mathematical properties of the arithmetic operations).

A language's user-defined types, on the other hand, are built from data structures. A data structure is not viewed as a simple value, but rather as a combination of simple values organized in a particular manner. A data structure is created from the language's built-in data types and the data structure's type constructors. The internal organization of a data structure is visible to the user, and it does not come with any operations other than the accessing operations of the data structure itself (such as the field selection operation on a record structure). One can, of course, define functions to operate on a data structure. However, in contrast to the standard procedure or function definitions available in most programming languages, these user-defined functions are not directly associated with the data type, and the implementation details of the data type and the operations are visible throughout the program.

It would be very desirable to have a mechanism in a programming language to construct data types that would have as many of the characteristics of a built-in type as possible. Such a mechanism should provide the following:

1. A method for defining both a data type and operations on that type. The definitions should all be collected in one place, and the operations should be directly associated with the type. The definitions should not depend on any implementation details. The definitions of the operations should include a specification of their semantics.
2. A method for collecting the implementation details of the type and its operations in one place, and of restricting access to these details by programs that use the data type.

A data type constructed using a mechanism satisfying one or both of these criteria is often called an **abstract data type** (or **ADT** for short). Note, however, that such a type is not really more abstract than an ordinary built-in type. It is just a more comprehensive way of creating user-defined types than the usual type declaration mechanism of Algol-like languages.

Criteria 1 and 2 promote three important design goals for data types: modifiability, reusability, and security. Modifiability is enhanced by interfaces that are implementation-independent, since changes can be made to an implementation without affecting its use by the rest of the program. Reusability is enhanced by standard interfaces, since the code can be reused by or plugged into different programs. Security is enhanced by protecting the implementation details from arbitrary modification by other parts of a program.

Some language authors dispense with criteria 1 and 2 in favor of encapsulation and information hiding, which they consider the essential properties of an abstract data type mechanism. **Encapsulation** refers to: a) the collection of all definitions related to a data type in one location; and b) restricting the use of the type to the operations defined at that location. **Information hiding** refers to the separation of implementation details from these definitions and the suppression of these details in the use of the data type. Since encapsulation and information hiding are sometimes difficult to separate in practice, we will instead focus on criteria 1 and 2 as our basis for studying abstract data types.

Confusion can sometimes result from the failure to distinguish a **mechanism** for constructing types in a programming language that has the foregoing properties, with the **mathematical concept** of a type, which is a conceptual model for actual types. This second notion is sometimes also called an abstract data type, or abstract type. Such mathematical models are often given in terms of an **algebraic specification**, which can be used to create an actual type in a programming language using an abstract data type mechanism with the foregoing properties.

Even more confusion exists over the difference between abstract data types and **object-oriented programming**, the subject of Chapter 6. Object-oriented programming emphasizes the capability of language entities to control their own use during execution and to share operations in carefully controlled ways. In an object-oriented programming language, the primary language entity is the object—that is, something that occupies memory and has state. Objects are also active, however, in that they control access to their own memory and state. In this sense, an abstract data type mechanism lends itself to the object-oriented approach, since information about a type is localized and access to this information is controlled. However, abstract data type mechanisms do not provide the level of active control that represents true object-oriented programming. See Section 11.7 and Chapter 6 for more details.

The notion of an abstract data type is in fact independent of the language paradigm (functional, imperative, object-oriented) used to implement it, and languages from all three paradigms are used in this chapter as examples of different approaches to ADTs.

An ADT mechanism is often expressed in terms of a somewhat more general concept called a **module**, which is a collection of services that may or may not include a data type or types. In the sections that follow, we describe a method for specifying abstract data types and then introduce some standard abstract data type mechanisms in programming languages. Next, we describe some aspects of modules and separate compilation, which we compare to abstract data types, with examples from C, C++, and Java. Additional sections provide examples from Ada and ML. We also survey some of the limitations of these mechanisms. In the last section, we discuss the mathematics of abstract data types.

11.1 The Algebraic Specification of Abstract Data Types

In this section, we focus on one example of an abstract data type, the **complex** data type, which is not a built-in data type in most programming languages. Mathematically, complex numbers are well-known objects and are extremely useful in practice, since they represent solutions to algebraic equations. A complex number is usually represented as a pair of real numbers (x, y) in Cartesian coordinates, which are conventionally written as $x + iy$, where i is a symbol representing the complex number $\sqrt{-1}$. x is called the "real" part, and y is called the "imaginary" part. There are also other ways to represent complex numbers—for example, as polar coordinates (r,θ). In defining complex numbers, however, we shouldn't need to specify their representation, but only the operations that apply to them. These include the usual arithmetic operations +, −, *, and /. We also need a way to create a complex number from a real and imaginary part, as well as functions to extract the real and imaginary parts from an existing complex number.

A general specification of a data type must include the name of the type and the names of the operations, including a specification of their parameters and returned values. This is the **syntactic specification** of an abstract data type and is often also called the **signature** of the type. In a language-independent specification, it is appropriate to use the function notation of mathematics for the operations of the data type: Given a function f from set X to set Y, X is the domain, Y is the range, and we write f: $X \rightarrow Y$. Thus, a signature for the complex data type looks like this:

type complex **imports** real

operations:

+: complex × complex → complex
-: complex × complex → complex
*: complex × complex → complex
/: complex × complex → complex
-: complex → complex
makecomplex: real × real → complex
realpart: complex → real
imaginarypart: complex → real

Note that the dependence of the complex data type on an already existing data type, namely, real, is made explicit by the "imports real" clause. (Do not confuse this imports clause with the `import` statement in some languages.) Note also that the negative sign is used for both subtraction and negation, which are two different operations. Indeed, the usual names or symbols are used for all the arithmetic operations. Some means must eventually be used to distinguish these from the arithmetic operations on the integers and reals (or they must be allowed to be overloaded, as some languages provide).

The preceding specification lacks any notion of semantics, or the properties that the operations must actually possess. For example, $z * w$ might be defined to be always 0! In mathematics, the semantic properties of functions are often described by **equations** or **axioms**. In the case of arithmetic operations, examples of axioms are the associative, commutative, and distributive laws. In the equations that follow, x, y, and z are assumed to be variables of type complex; that is, they can take on any complex value.

$x + (y + z) = (x + y) + z$ (associativity of +)
$x * y = y * x$ (commutativity of *)
$x * (y + z) = x * y + x * z$ (distributivity of * over +)

Axioms such as these can be used to define the semantic properties of complex numbers, or the properties of the complex data type can be **derived** from those of the real data type by stating properties of the operations that lead back to properties of the real numbers. For example, complex addition can be based on real addition according to the following properties:

realpart(x + y) = realpart(x) + realpart(y)
imaginarypart(x + y) = imaginarypart(x) + imaginarypart(y)

The appropriate arithmetic properties of the complex numbers can then be proved from the corresponding properties for reals. A complete algebraic specification of type complex combines signature, variables, and equational axioms:[1]

type complex **imports** real

operations:

+: complex × complex → complex
=: complex × complex → complex
*: complex × complex → complex
/: complex × complex → complex
-: complex → complex
makecomplex : real × real → complex
realpart : complex → real
imaginarypart : complex → real

variables: x,y,z: complex; r,s: real

axioms:

realpart(makecomplex(r,s)) = r
imaginarypart(makecomplex(r,s)) = s
realpart(x + y) = realpart(x) + realpart(y)
imaginarypart(x + y) = imaginarypart(x) + imaginarypart(y)
realpart(x - y) = realpart(x) - realpart(y)
imaginarypart(x - y) = imaginarypart(x) - imaginarypart(y)
. . .
(more axioms)
. . .

Such a specification is called an **algebraic specification** of an abstract data type. It provides a concise specification of a data type and its associated operations. The accompanying equational semantics give a clear indication of implementation behavior, often containing enough information to allow coding directly from the equations. Finding an appropriate set of equations, however, can be a difficult task. The next part of this section gives some suggestions on how to find them.

Remember the difference in the foregoing specification between **equality** as used in the axioms and the **arrow** of the syntactic specification of the functions. Equality is of values returned by functions, while the arrows separate a function's domain and range.

[1]The variables section of a specification is a notational convenience that simply lists the names (and their types) over which the axioms are assumed to be universally quantified in the mathematical sense. Thus, the first axiom in this algebraic specification is actually "for all real r and s, realpart(makecomplex(r,s)) = s."

A second example of an algebraic specification of an abstract data type is the following specification of a queue:

type queue(element) **imports** boolean

operations:

createq: queue
enqueue: queue × element → queue
dequeue: queue → queue
frontq: queue → element
emptyq: queue → boolean

variables: *q*: queue; *x*: element

axioms:

emptyq(createq) = true
emptyq(enqueue(*q*,*x*)) = false
frontq(createq) = error
frontq(enqueue(*q*,*x*)) = if emptyq(*q*) then *x* else frontq(*q*)
dequeue(createq) = error
dequeue(enqueue(*q*,*x*)) = if emptyq(*q*) then *q* else enqueue(dequeue(*q*),*x*)

This specification exhibits several new features. First, the data type queue is **parameterized** by the data type element, which is left unspecified. Such a type parameter can be replaced by any type. You indicate it by placing its name inside parentheses, just as a function parameter would be. Second, the specification includes a constant, create, that is technically not a function. Such constants may also be a part of the algebraic specification; if desired, createq can be viewed as a function of no parameters that always returns the same value. Intuitively, we can see that createq is a new queue that has been initialized to empty. Third, we now include axioms that specify error values, such as

frontq(createq) = error

Such axioms can be called **error axioms**, and they provide limitations on the application of the operations. The actual error value of an error axiom is unspecified. Finally, the equations are specified using an if-then-else function, whose semantics are as follows:

if true then *a* else *b* = *a*
if false then *a* else *b* = *b*

Note also that the dequeue operation as specified does not return the front element of the queue, as in most implementations: It simply throws it away. Abstractly, this is simpler to handle and does not take away any functionality of the queue, since the frontq operation can extract the front element before a dequeue operation is performed.

The equations specifying the semantics of the operations in an algebraic specification of an abstract data type can be used as a specification of the properties of an implementation. It can also be used as a guide to the code for an implementation, and to prove specific properties about objects of the type.

For example, the following applications of axioms show that dequeuing from a queue with one element leaves an empty queue:

```
emptyq(dequeue(enqueue (createq, x)))
        = emptyq(createq)        (by the sixth axiom above)
        = true                   (by the first axiom)
```

Note that the operations and the axioms are specified in a mathematical form that includes no mention of memory or of assignment. Indeed, enqueue returns a newly constructed queue every time a new element is added. These specifications are in fact in purely functional form, with no variables and no assignment. (The variables in the specification are taken in the mathematical sense to be just names for values.) This is important, since mathematical properties, such as the semantic equations, are much easier to express with purely functional constructions. (See Chapter 4 for more on purely functional programming.) In practice, abstract data type implementations often replace the specified functional behavior with an equivalent imperative one using explicit memory allocation, as well as assignment or update operations. The question of how to infer that an imperative implementation is in some sense equivalent to a purely functional one, such as the above, is beyond the scope of our study here.

How do we find an appropriate axiom set for an algebraic specification? In general, this is a difficult question. We can, however, make some judgments about the kind and number of axioms necessary by looking at the syntax of the operations. An operation that creates a new object of the data type being defined is called a **constructor**, while an operation that retrieves previously constructed values is called an **inspector**. In the queue example, createq and enqueue are constructors, while frontq, dequeue, and emptyq are inspectors. Inspector operations can also be broken down into **predicates**, which return Boolean values, and **selectors**, which return non-Boolean values. Thus frontq and dequeue are selectors,[2] while emptyq is a predicate.

In general, we need one axiom for each combination of an inspector with a constructor. For the queue example, the axiom combinations are:

```
emptyq(createq)
emptyq(enqueue(q,x))
frontq(createq)
frontq(enqueue(q,x))
dequeue(createq)
dequeue(enqueue(q,x))
```

According to this scheme, there should be six rules in all, and that is in fact the case.

As a final example of an algebraic specification of an abstract data type, we give a specification for the stack abstract data type, which has many of the same features as the queue ADT:

type stack(element) **imports** boolean

operations:

createstk: stack
push: stack × element → stack
pop: stack → stack
top: stack → element
emptystk: stack → boolean

[2]Strictly speaking, dequeue is a non-primitive constructor. We call it a selector because it acts like one in this context.

variables: *s*: stack; *x*: element

axioms:

emptystk(createstk) = true
emptystk(push(*s*,*x*)) = false
top(createstk) = error
top(push(*s*,*x*)) = *x*
pop(createstk) = error
pop(push(*s*,*x*)) = *s*

In this case, the operations createstk and push are constructors, the pop and top operations are selectors, and the emptystk operation is a predicate. By our proposed scheme, therefore, there should again be six axioms.

11.2 Abstract Data Type Mechanisms and Modules

In this section, we explore some mechanisms for creating abstract data types and the manner in which modules represent an extension of these mechanisms.

11.2.1 Abstract Data Type Mechanisms

Some languages have a specific mechanism for expressing abstract data types. Such a mechanism must have a way of separating the specification or signature of the ADT (the name of the type being specified, and the names and types of the operations) from its implementation (a data structure implementing the type and a code body for each of the operations). Such a mechanism must also guarantee that any code outside the ADT definition cannot use details of the implementation but can operate on a value of the defined type only through the provided operations.

ML is an example of a language with a special ADT mechanism, called `abstype`. It is viewed as a kind of type definition mechanism. In Figure 11.1 it is used to specify a queue.

```
(1) abstype 'element Queue = Q of 'element list
(2) with
(3)  val createq = Q [];
(4)  fun enqueue (Q lis, elem) = Q (lis @ [elem]);
(5)  fun dequeue (Q lis) = Q (tl lis);
(6)  fun frontq (Q lis) = hd lis;
(7)  fun emptyq (Q []) = true | emptyq (Q (h::t)) = false;
(8) end;
```

Figure 11.1 A queue ADT as an ML `abstype`, implemented as an ordinary ML list

Here we have used an ordinary list in ML (`'element list`, line 1). Since an `abstype` must be a new type and not a type defined elsewhere, we also had to wrap `'element list` in a constructor, for which we have used the single letter `Q`;[3] this constructor is not visible outside the definition of the `abstype`. Recall also that lists are written in ML using square brackets `[...]`, `@` is the append operator (line 4), `h::t` is a pattern for the list with head `h` and tail `t` (line 7), `hd` is the head function on lists (line 6), and `tl` is the tail function on lists (line 5). (For more on the details of the actual type constructor specification `Q of 'element list` see Chapter 9.)

When this definition is processed by the ML translator, it responds with a description of the signature of the type:

```
type 'a Queue
val createq = - : 'a Queue
val enqueue = fn : 'a Queue * 'a -> 'a Queue
val dequeue = fn : 'a Queue -> 'a Queue
val frontq = fn : 'a Queue -> 'a
val emptyq = fn : 'a Queue -> bool
```

Since ML has parametric polymorphism, the `Queue` type can be parameterized by the type of the element that is to be stored in the queue, and we have done that with the type parameter `'element` (reported as `'a` by the ML system). Notice how ML refuses to specify the internal structure in this description, writing a dash for the actual representation of `createq`, and using only `'a Queue` to refer to the type. Thus, all internal details of the representation of `Queue`, including its constructors, are suppressed and remain private to the code of the `abstype` definition.

With the above definition, the `Queue` type can be used in ML as follows:

```
- val q = enqueue(createq,3);
val q = - : int Queue
- val q2 = enqueue(q,4);
val q2 = - : int Queue
- frontq q2;
val it = 3 : int
- val q3 = dequeue q2;
val q3 = - : int Queue
- frontq q3;
val it = 4 : int
```

[3]It is a common practice to use the same name as the data type, namely `Queue`, for this constructor, but we have not done so in an attempt to make the code easier to understand.

As a second example, a complex number ADT can be specified in ML as shown in Figure 11.2.

```
(1) abstype Complex = C of real * real
(2) with
(3)  fun makecomplex (x,y) = C (x,y);
(4)  fun realpart (C (r,i)) = r;
(5)  fun imaginarypart (C (r,i)) = i;
(6)  fun +: ( C (r1,i1), C (r2,i2) ) = C (r1+r2, i1+i2);
(7)  infix 6 +: ;
(8)  (* other operations *)
(9) end;
```

Figure 11.2 A complex number ADT as an ML `abstype`

ML allows user-defined operators, which are called **infix functions** in ML. Infix functions can use special symbols, but they cannot reuse the standard operator symbols, since ML does not allow user-defined overloading. Thus, in line 6 of Figure 11.2, we have defined the addition operator on complex numbers to have the name `+:` (a plus sign followed by a colon). Line 7 makes this into an infix operator (with left associativity) and gives it the precedence level 6, which is the ML precedence level for the built-in additive operators such as `+` and `-`. The `Complex` type can be used as follows:

```
- val z = makecomplex (1.0,2.0);
val z = - : Complex
- val w = makecomplex (2.0,~1.0); (* ~ is negation *)
val w = - : Complex
- val x = z +: w;
val x = - : Complex
- realpart x;
val it = 3.0 : real
- imaginarypart x;
val it = 1.0 : real
```

11.2.2 Modules

An ADT mechanism such as we have described in Section 11.2.1—essentially an extension of the definition of an ordinary data type—is completely adequate, even superior, as a way of implementing an abstract data type in a language with strong type abstractions such as ML. However, even in ML it is not the end of the story, since a pure ADT mechanism does not address the entire range of situations where an ADT-like abstraction mechanism is useful in a programming language.

Consider, for example, a mathematical function package consisting of the standard functions such as sine, cosine, exponential, and logarithmic functions. Because these functions are closely related, it makes sense to encapsulate their definitions and implementations and hide the implementation details (which could then be changed without changing client code that uses these functions). However, such

a package is not associated directly with a data type (or the data type, such as `double`, is built-in or defined elsewhere). Thus, such a package does not fit the format of an ADT mechanism as described in Section 11.2.1.

Similarly, as shown in Figure 11.3, a programming language compiler is a large program that is typically split into separate pieces corresponding to the phases described in Chapter 1.

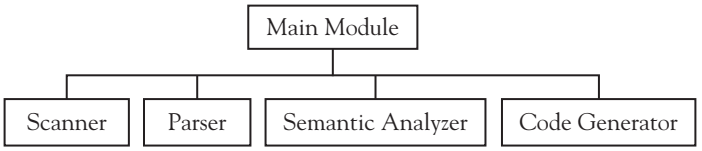

Figure 11.3 Parts of a programming language compiler

We would certainly wish to apply the principles of encapsulation and information hiding to each of the code pieces in this diagram. Here also, there is no special data type that can be associated directly with each phase. Instead, these examples demonstrate the need for an encapsulation mechanism that is viewed more generally as a provider of services, and that is given by the concept of a module:

DEFINITION: A ***module*** is a program unit with a public interface and a private implementation; all services that are available from a module are described in its public interface and are exported to other modules, and all services that are needed by a module must be imported from other modules.

As providers of services, modules can export any mix of data types, procedures, variables, and constants. Also, because modules have explicit (public) interfaces and separate (private) implementations, they are ideal mechansims to provide separate compilation and library facilities within a software development environment. Indeed, many languages, such as Ada and Java, tie the structure of modules to separate compilation issues (although the relationship is kept loose enough to allow for variation from system to system). Typically in a compiler-based system, the interface for each module in a library or program is kept in textual form, while the implementation may only be provided as object code and is only needed at the link step to produce a running program.

Thus, modules are an essential tool in program decomposition, complexity control, and the creation of libraries for code sharing and reuse. Additionally, modules assist in the control of **name proliferation**: large programs have large numbers of names, and mechanisms to control access to these names, as well as preventing name clashes, are an essential requirement of a modern programming language. Nested scopes are only the first step in such name control, and modules typically provide additional scope features that make the task of name control more manageable. Indeed, one view of a module is that its main purpose is to provide controlled scopes, exporting only those names that its interface requires and keeping hidden all others. At the same time, even the exported names should be **qualified** by the module name to avoid accidental name clashes when the same name is used by two different modules. Typically, this is done by using the same dot notation used for structures, so that feature `y` from module `X` has the name `X.y` when imported into client code. (Note that the ML `abstype` mechanism discussed previously does not have this essential name control feature.)

Finally, a module mechanism can document the dependencies of a module on other modules by requiring explicit import lists whenever code from other modules is used. These dependencies can be used by a compiler to automatically recompile out-of-date modules and to automatically link in separately compiled code.

In the following sections, we describe the mechanisms that several different languages (with very different approaches) offer to provide modular facilities.

11.3 Separate Compilation in C, C++ Namespaces, and Java Packages

We collect three different mechanisms in this section, because they are all less concerned with the full range of modular properties and are aimed primarily at separate compilation and name control.

11.3.1 Separate Compilation in C and C++

The first language we consider is C. C does not have any module mechanisms as such, but it does have separate compilation and name control features that can be used to at least simulate modules in a reasonably effective way. Consider, for example, a queue data structure in C. A typical organization would be to place type and function specifications in a so-called **header file** `queue.h`, as in Figure 11.5. Only type definitions and function declarations without bodies (called **prototypes** in C) go into this file. This file is used as a specification of the queue ADT by textually including it in client code as well as implementation code using the C preprocessor `#include` directive. Sample implementation code and client code are provided in Figures 11.6 and 11.7, respectively. Graphically, the separation of specification, implementation, and client code is as shown in Figure 11.4 (with the inclusion dependencies represented by the arrows).

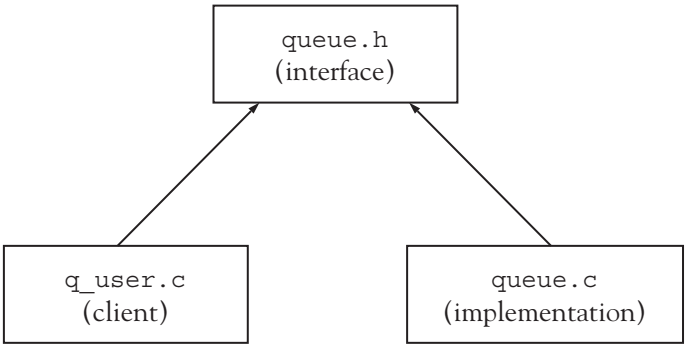

Figure 11.4 Separation of specification, implementation, and client code

```
(1) #ifndef QUEUE_H
(2) #define QUEUE_H

(3) struct Queuerep;
(4) typedef struct Queuerep * Queue;
```

Figure 11.5 A `queue.h` header file in C *(continues)*

(continued)

```
(5) Queue createq(void);
(6) Queue enqueue(Queue q, void* elem);
(7) void* frontq(Queue q);
(8) Queue dequeue(Queue q);
(9) int emptyq(Queue q);

(10) #endif
```

Figure 11.5 A `queue.h` header file in C

```
(1) #include "queue.h"
(2) #include <stdlib.h>
(3)  struct Queuerep
(4)  { void* data;
(5)    Queue next;
(6)  };

(7)  Queue createq()
(8)  { return 0;
(9)  }

(10) Queue enqueue(Queue q, void* elem)
(11) { Queue temp = malloc(sizeof(struct Queuerep));
(12)   temp->data = elem;
(13)   if (q)
(14)   { temp->next = q->next;
(15)     q->next = temp;
(16)     q = temp;
(17)   }
(18)   else
(19)   { q = temp;
(20)     q->next = temp;
(21)   }
(22)   return q;
(23) }

(24) void* frontq(Queue q)
(25) { return q->next->data;
(26) }

(27) Queue dequeue(Queue q)
(28) { Queue temp;
(29)   if (q == q->next)
```

Figure 11.6 A `queue.c` implementation file in C *(continues)*

(continued)

```
(30)   { temp = q;
(31)     q = 0;
(32)   }
(33)   else
(34)   { temp = q->next;
(35)     q->next = q->next->next;
(36)   }
(37)   free(temp);
(38)   return q;
(39) }

(40) int emptyq(Queue q)
(41) { return q == 0;
(42) }
```

Figure 11.6 A `queue.c` implementation file in C

```
(1)  #include <stdlib.h>
(2)  #include <stdio.h>
(3)  #include "queue.h"

(4)   main()
(5)   {  int *x = malloc(sizeof(int));
(6)      int *y = malloc(sizeof(int));
(7)      int *z;
(8)      Queue q = createq();
(9)      *x = 2;
(10)     *y = 3;
(11)     q = enqueue(q,x);
(12)     q = enqueue(q,y);
(13)     z = (int*) frontq(q);
(14)     printf("%d\n",*z); /* prints 2 */
(15)     q = dequeue(q);
(16)     z = (int*) frontq(q);
(17)     printf("%d\n",*z); /* prints 3 */
(18)     return 0;
(19)  }
```

Figure 11.7 A client code file `q_user.c` in C

We make the following comments about the code shown in Figures 11.5–11.7.

The header file `queue.h` exhibits the standard C convention of surrounding the code with an `#ifndef ... #endif` preprocessor block (Figure 11.5, lines 1, 2, and 10). This causes the symbol `QUEUE_H` (a conventional modification of the filename) to be defined the first time the file is included, and all repeated inclusions to be skipped. This is a necessary feature of header files, since there may be multiple paths through which a header file is included. Without it, the resulting repeated `Queue` type definition will cause a compilation error.

The definition of the `Queue` data type is hidden in the implementation by defining `Queue` to be a pointer type (Figure 11.5, line 4), and leaving the actual queue representation structure (`Queuerep`, line 3) as an **incomplete type**. If this were not done, then the entire `Queue` data structure would have to be declared in the header file (since the compiler would need to know its size) and, thus, exposed to clients. Clients would then be able to write code that depends on the implementation, thus destroying the desired implementation independence of the interface. On the other hand, making `Queue` into a pointer type has other drawbacks, since assignment and equality tests do not necessarily work as desired—see Section 11.7.

Ideally, a `Queue` data type needs be parameterized by the data type of the elements it stores, as you have seen. However, C does not have user polymorphism of any kind. Nevertheless, parametric polymorphism can be effectively simulated by the use of `void*` elements, which are "pointers to anything" in C and which can be used with casts to store any dynamically allocated object. Of course, there is the added burden of dynamically allocating all data and providing appropriate casts. Also, this code can be dangerous, because client code must always know the actual data type of the elements being stored.

The `queue.c` implementation file provides the actual definition for `Queuerep` (Figure 11.6, lines 3–6), and the code for the bodies of the interface functions. In the sample code, we have used a singly linked circular list with the queue itself represented by a pointer to the last element (the empty queue is represented by the null pointer 0).

The files `queue.c` and `q_user.c` can be separately compiled, and either can be changed without recompiling the other. For example, the queue implementation could be changed to a doubly linked list with pointers to the first and last elements (Exercise 11.9) without affecting any client code. Also, after the `queue.c` file is compiled, no further recompilation is needed (and the implementation can be supplied to clients as header and object file only). This makes the implementation and client code reasonably independent of each other.

This mechanism for implementing abstract data types works fairly well in practice; it is even better suited to the definition of modules that do not export types, such as a math module (see the standard C `math.h` file), where the complications of incomplete types and dynamic allocation do not arise. However, the effectiveness of this mechanism depends entirely on convention, since neither C compilers nor standard linkers enforce any of the protection rules normally associated with module or ADT mechanisms. For example, a client could fail to include the header file `queue.h`, but simply textually copy the definitions in `queue.h` directly into the `q_user.c` file (by cut and paste, for instance). Then the dependence of the client code on the queue code is not made explicit by the `#include` directive. Worse, if changes to the `queue.h` file were made (which would mean all client code should be recompiled), these changes would not automatically be imported into the client code, but neither the compiler nor the linker would report an error: not the compiler, since the client code would be consistent with the now-incorrect queue interface code, and not the linker, since the linker only ensures that some definition exists for every name, not that the definition is consistent with its uses. In fact, client code can circumvent the protection of the incomplete `Queuerec` data type itself by this same mechanism: If the actual structure of

`Queuerec` is known to the client, it can be copied into the client code and its structure used without any complaint on the part of the compiler or linker. This points out that this use of separate compilation is not a true module mechanism but a simulation, albeit a reasonably effective one.

We note also that, while the `#include` directives in a source code file do provide some documentation of dependencies, this information cannot be used by a compiler or linker because there are no language rules for the structure of header files or their relationship to implementation files. Thus, all C systems require that the user keep track of out-of-date source code and manually recompile and relink.[4]

One might suppose that C++ improves significantly on this mechanism, and in some ways it does. Using the class mechanism of object-oriented programming, C++ allows better control over access to the `Queue` data type. C++ also offers better control over names and name access through the `namespace` mechanism (next subsection), but C++ offers no additional features that would enhance the use of separate compilation to simulate modules.[5] In fact, such additional features would, as noted, place an extra burden on the linker to discover incompatible definitions and uses, and a strict design goal of C++ was that the standard C linker should continue to be used (Stroustrup [1994], page 120). Thus, C++ continues to use textual inclusion and header files to implement modules and libraries.

One might also imagine that the C++ template mechanism could be used to eliminate the use of `void*` to simulate parametric polymorphism in C. Unfortunately, templates in C++ do not mix well with separate compilation, since instantiations of template types require knowledge of the separately compiled template code by the C++ compiler. Thus, the use of templates would not only force the details of the `Queuerec` data type to be included in the header file, but, using most compilers, even the code for the function implementations would have to be put into the header file (and thus the entire contents of `queue.c` would have to be moved to `queue.h`!). In fact, the standard template library of C++ exhibits precisely this property. Virtually all the code is in the header files only. Fortunately, the use of access control within the class mechanism still allows some form of protection, but the situation is far from ideal. The use of templates in data types is, thus, most effective in conjunction with the class construct. See Chapter 6 for an example.

11.3.2 C++ Namespaces and Java Packages

The `namespace` mechanism in C++ provides effective support for the simulation of modules in C. The `namespace` mechanism allows the programmer to introduce a named scope explicitly, thus avoiding name clashes among separately compiled libraries. (However, the use of namespaces is not explicitly tied to separate compilation.)

Consider the previous example of a Queue ADT structure in C. A major problem with the implementation of this ADT is that other libraries are likely available in a programming environment that could well include a different implementation of a queue, but with the same name. Additionally, a programmer may want to include some code from that library as well as use the given Queue implementation. In C this is not possible. In C++ a namespace can be used to disambiguate such name clashes. Figure 11.8 lists the code for the `queue.h` file as it would look using a namespace in C++.

[4]The lack of such a module facility in C led directly to the construction of tools to manage compilation and linking issues, such as the UNIX `make` utility.

[5]Strictly speaking, the C++ standard does make it a violation of the "one definition rule" to have two different definitions of the same data structure that are not syntactically and semantically identical, but then allows linkers to fail to catch this error. See Stroustrup [1997], Section 11.2.3 for a discussion.

```
#ifndef QUEUE_H
#define QUEUE_H

namespace MyQueue
{ struct Queuerep;
  typedef Queuerep * Queue;
            // struct no longer needed in C++
  Queue createq();
  Queue enqueue(Queue q, void* elem);
  void* frontq(Queue q);
  Queue dequeue(Queue q);
  bool emptyq(Queue q);
}

#endif
```

Figure 11.8 The `queue.h` header file in C++ using a namespace

Inside the `queue.cpp` file, the definitions of the queue functions must now use the scope resolution operator, for example:

```
struct MyQueue::Queuerep
{ void* data;
  Queue next;
};
```

Alternatively, one could reenter the `MyQueue` namespace by surrounding the implementation code in `queue.cpp` with the same namespace declaration as in the header. (Namespaces can be reentered whenever one wishes to add definitions or implementations to a namespace.)

A client of `MyQueue` has three options:

1. After the appropriate `#include`, the user can simply refer to the `MyQueue` definitions using the qualified name (i.e., with `MyQueue::` before each name used from the `MyQueue` namespace).
2. Alternatively, the user can write a `using` declaration for each name used from the namespace and then use each such name without qualification.
3. Finally, the user can "unqualify" all the names in the namespace with a single `using namespace` declaration.

Consider Figure 11.9, which lists C++ client code for the above C++ queue code. This code exhibits all three of the above options to reference names in a namespace. A `using` declaration (option 2) is given on line 3, allowing the standard library function `endl` from `iostream` to be used without qualification. A `using namespace` declaration (option 3) is given on line 4, allowing all names in `MyQueue` to be

used without qualification, and the qualified name `std::cout` is used for the references to `cout` (lines 18 and 21).

```
(1)  #include <iostream>
(2)  #include "queue.h"

(3)  using std::endl;
(4)  using namespace MyQueue;

(5)  main()
(6)  {  int *x = new int;
(7)     int *y = new int;
(8)     int *z;
(9)     // explicit qualification unnecessary
(10)    // but permitted
(11)    Queue q = MyQueue::createq();
(12)    *x = 2;
(13)    *y = 3;
(14)    q = enqueue(q,x);
(15)    q = enqueue(q,y);
(16)    z = (int*) frontq(q);
(17)    // explicit qualification needed for cout
(18)    std::cout << *z << endl; // prints 2
(19)    q = dequeue(q);
(20)    z = (int*) frontq(q);
(21)    std::cout << *z << endl; // prints 3
(22) }
```

Figure 11.9 A client code file `q_user.cpp` in C++ with `using` declarations

Java, too, has a namespace-like mechanism, called the **package**. Since Java emphasizes object orientation and the class construct, packages are constructed as groups of related classes. Each separately compiled file in a Java program is allowed to have only one public class, and this class can be placed in a package by writing a package declaration as the first declaration in the source code file:

```
package myqueue;
```

Other Java code files can now refer to the data type `Queue` by the name `myqueue.Queue`, if the data type `Queue` is the public class in its file. The use of a Java `import` declaration also allows the programmer to dereference this name in a similar manner to the C++ `using` declaration:

```
import myqueue.Queue;
```

The use of an asterisk after the package name allows the programmer to import *all* names in a Java package:

```
import myqueue.*;
```

This is similar to the C++ `using namespace` declaration.

Note here that the Java `import` declaration does *not* correspond to the abstract notion of an import in the definition of a module in the previous section. Indeed, an import in the latter sense represents a dependency on external code, and it is more closely related to the C `#include` statement than to the Java `import` declaration. Code dependencies in Java are hidden by the fact that the compiler automatically searches for external code using the package name and, typically, a system search path (commonly called `CLASSPATH`). Package names can also contain extra periods, which are usually interpreted by Java as representing subdirectories. For example, the package `myqueue` might be best placed in a directory `mylibrary`, and then the package name would be `mylibrary.myqueue`. Using the fully qualified name of a library class allows any Java code to access any other public Java code that is locatable along the search path by the compiler, without use of an `import` declaration. Indeed, the compiler will also check for out-of-date source code files, and recompile all dependent files automatically. Thus, library dependencies can be well hidden in Java. Even the link step in Java does not need any dependency information, since linking is performed only just prior to execution by a utility called the **class loader**, and the class loader uses the same search mechanism as the compiler to find all dependent code automatically.

11.4 Ada Packages

Ada's module mechanism is the **package** (briefly discussed in Chapter 9), because the package is used in Ada not only to implement modules but also to implement parametric polymorphism.[6] An Ada package is divided into a package **specification** and a package **body**. A package specification is the public interface to the package, and it corresponds to the syntax or signature of an ADT (Section 11.1). Package specifications and package bodies also represent compilation units in Ada, and can be compiled separately.[7] Clients are also compiled separately and linked to compiled package bodies. A package specification in Ada for type `Complex` is given in Figure 11.10.

```
(1)  package ComplexNumbers is

(2)  type Complex is private;

(3)  function "+"(x,y: in Complex) return Complex;
(4)  function "-"(x,y: in Complex) return Complex;
(5)  function "*"(x,y: in Complex) return Complex;
(6)  function "/"(x,y: in Complex) return Complex;
(7)  function "-"(z: in Complex) return Complex;
(8)  function makeComplex (x,y: in Float) return Complex;
(9)  function realPart (z: in Complex) return Float;
```

Figure 11.10 A package specification for complex numbers in Ada *(continues)*

[6]Ada packages should not be confused with Java packages, which are a rather different concept; see the previous section.
[7]Some implementations may not require the compilation of a package specification, since the compiled version is essentially a symbol table with the same information as the specification file itself.

(continued)

```
(10) function imaginaryPart (z: in Complex) return Float;

(11) private
(12)    type Complex is
(13)    record
(14)       re, im: Float;
(15)    end record;
(16) end ComplexNumbers;
```

Figure 11.10 A package specification for complex numbers in Ada

This specification of a complex number ADT uses oveloaded operators in Ada. Additionally, this specification uses a **private** declaration section (lines 11–16). Any declarations given in this section are inaccessible to a client. Type names, however, can be given in the public part of a specification and designated as private (line 2). Note, however, that an actual type declaration must still be given in the private part of the specification. This violates both criteria for abstract data type mechanisms. Specifically, it violates criterion 1 in that the specification is still dependent on actual implementation details, and it violates criterion 2 in that the implementation details are divided between the specification and the implementation. However, the use of the private declarations at least prevents clients from making any use of the implementation dependency, although clients still must be recompiled if the implementation type changes. As in the C queue example of Section 11.2, this problem can be partially removed in Ada by using pointers, and a revised specification in Ada that removes the dependency of the specification on the implementation type (at the cost of introducing pointers) is as follows:

```
package ComplexNumbers is

type Complex is private;

-- functions as before

private
   type ComplexRecord;
   -- incomplete type defined in package body
   type Complex is access ComplexRecord;
   -- a pointer type
end ComplexNumbers;
```

A corresponding implementation for this somewhat improved package specification is given in Figure 11.11.

```
(1)  package body ComplexNumbers is

(2)  type ComplexRecord is record
(3)     re, im: Float;
(4)  end record;

(5)  function "+"(x,y: in Complex) return Complex is
(6)  t: Complex;
(7)  begin
(8)     t := new ComplexRecord;
(9)     t.re := x.re + y.re;
(10)    t.im := x.im + y.im;
(11)    return t;
(12) end "+";

(13) -- more operations here

(14) function makeComplex (x,y: in Float)
(15)       return Complex is
(16) begin
(17)    return new ComplexRecord'(re => x , im => y);
(18) end makeComplex;

(19) function realPart (z: in Complex) return Float is
(20) begin
(21)    return z.re;
(22) end realPart;
(23) function imaginaryPart (z: in Complex)
(24)    return Float is
(25) begin
(26)    return z.im;
(27) end imaginaryPart;
(28) end ComplexNumbers;
```

Figure 11.11 A package implementation for complex numbers in Ada using pointers

Note that pointers are automatically dereferenced in Ada by the dot notation. Also, note that fields can be assigned during allocation using the apostrophe attribute delimiter (the single quotation mark in `makeComplex`, line 17). A client program can use the `ComplexNumbers` package by including a `with` clause at the beginning of the program, similar to a C `#include` directive:

```
with ComplexNumbers;

procedure ComplexUser is
  z,w: ComplexNumbers.Complex;
begin
  z := ComplexNumbers.makeComplex(1.0,-1.0);
  ...
  w := ComplexNumbers."+"(z,z);
end ComplexUser;
```

Packages in Ada are automatically namespaces in the C++ sense, and all referenced names must be referred to using the package name as a qualifier (the notation, however, is identical to field dereference in records or structures). Ada also has a `use` declaration analogous to the `using` declaration of C++ that dereferences the package name automatically:

```
with ComplexNumbers;
use ComplexNumbers;
procedure ComplexUser is
  z,w: Complex;
  ...
begin
  z := makeComplex(1.0,-1.0);
  ...
  w := z + z;
  ...
end ComplexUser;
```

Notice that the infix form of the overloaded "+" operator can only be used when the package name is dereferenced.

One of the major uses for the package mechanism in Ada is to implement parameterized types as discussed in Chapter 9; parameterized packages are called **generic packages** in Ada. As an example, we give an implementation of a Queue ADT that is entirely analogous to the C implementation of Section 11.2 (a queue given by a circularly linked list with rear pointer). Figure 11.12 shows the package specification, Figure 11.13 shows the package implementation, and Figure 11.14 shows some sample client code.

```
(1)  generic
(2)    type T is private;
(3)  package Queues is
(4)    type Queue is private;
(5)    function createq return Queue;
(6)    function enqueue(q:Queue;elem:T) return Queue;
```

Figure 11.12 A parameterized queue ADT defined as an Ada generic package specification *(continues)*

(continued)

```
(7)    function frontq(q:Queue) return T;
(8)    function dequeue(q:Queue) return Queue;
(9)    function emptyq(q:Queue) return Boolean;
(10) private
(11)   type Queuerep;
(12)   type Queue is access Queuerep;
(13) end Queues;
```

Figure 11.12 A parameterized queue ADT defined as an Ada generic package specification

```
(1)  package body Queues is

(2)    type Queuerep is
(3)    record
(4)      data: T;
(5)      next: Queue;
(6)    end record;

(7)    function createq return Queue is
(8)    begin
(9)      return null;
(10)   end createq;

(11)   function enqueue(q:Queue;elem:T) return Queue is
(12)     temp: Queue;
(13)   begin
(14)     temp := new Queuerep;
(15)     temp.data := elem;
(16)     if (q /= null) then
(17)        temp.next := q.next;
(18)        q.next := temp;
(19)     else
(20)        temp.next := temp;
(21)     end if;
(22)     return temp;
(23)   end enqueue;
(24)   function frontq(q:Queue) return T is
(25)   begin
(26)     return q.next.data;
(27)   end frontq;
```

Figure 11.13 Generic package implementation in Ada for the `Queues` package specification of Figure 11.12 *(continues)*

(continued)

```
(28)   function dequeue(q:Queue) return Queue is
(29)   begin
(30)     if q = q.next then
(31)        return null;
(32)     else
(33)        q.next := q.next.next;
(34)        return q;
(35)     end if;
(36)   end dequeue;

(37)   function emptyq(q:Queue) return Boolean is
(38)   begin
(39)     return q = null;
(40)   end emptyq;

(41) end Queues;
```

Figure 11.13 Generic package implementation in Ada for the `Queues` package specification of Figure 11.12

```
(1)  with Ada.Text_IO; use Ada.Text_IO;

(2)  with Ada.Float_Text_IO;
(3)  use Ada.Float_Text_IO;

(4)  with Ada.Integer_Text_IO;
(5)  use Ada.Integer_Text_IO;

(6)  with Queues;

(7)  procedure Quser is

(8)  package IntQueues is new Queues(Integer);
(9)  use IntQueues;

(10) package FloatQueues is new Queues(Float);
(11) use FloatQueues;

(12) fq: FloatQueues.Queue := createq;
(13) iq: IntQueues.Queue := createq;

(14) begin
(15)   fq := enqueue(fq,3.1);
```

Figure 11.14 Sample Ada client code that uses the `Queues` generic package of Figures 11.12 and 11.13 *(continues)*

(continued)

```
(16)   fq := enqueue(fq,2.3);
(17)   iq := enqueue(iq,3);
(18)   iq := enqueue(iq,2);
(19)   put(frontq(iq)); -- prints 3
(20)   new_line;
(21)   fq := dequeue(fq);
(22)   put(frontq(fq)); -- prints 2.3
(23)   new_line;
(24) end Quser;
```

Figure 11.14 Sample Ada client code that uses the `Queues` generic package of Figures 11.12 and 11.13

We note the following about the code in Figure 11.14. First, lines 1–5 import and dereference standard Ada library IO functions. Line 6 imports the `Queues` package. Lines 8 and 10 create two different instantiations of the `Queues` generic package, one for integers and one for floating-point numbers; these are given the package names `IntQueues` and `FloatQueues`. These two packages are then dereferenced by the `use` declarations in lines 9 and 11. Note that this creates name ambiguities, since all of the names from each of these packages conflict with the names from the other package. Ada's overloading mechanism can, however, resolve most of these ambiguities, based on the data types of the parameters and results. Thus, it is still possible to use the simple names, such as `enqueue`, instead of the qualified names `IntQueues .enqueue` and `FloatQueues.enqueue`. However, it is impossible to disambiguate the names when the data type `Queue` is used in the definitions of `fq` and `iq` (lines 12 and 13). In this case, the qualified names `IntQueues.Queue and FloatQueues.Queue` are used to provide the necessary disambiguation.

11.5 Modules in ML

In Section 11.2, we demonstrated the ML `abstype` definition, which allows easy definition of abstract types with associated operations. ML also has a more general module facility, which consists of three mechanisms: **signatures**, **structures**, and **functors**. The signature mechanism is essentially an interface definition (it even adopts the name *signature* from the mathematical description of the syntax of an ADT as in Section 11.1). A structure is essentially an implementation of a signature; viewed another way, the signature is essentially the type of the structure. Functors are essentially functions from structures to structures, with the structure parameters having "types" given by signatures. Functors allow for the parameterization of structures by other structures. For more on functors, see Section 11.7.4.

Figure 11.15 shows an ML signature for a queue ADT. The `sig ... end` expression (lines 2 and 9) produces a value of type `signature` by listing the types of the identifiers being defined by the signature. This signature value is then stored as the name `QUEUE`. Since a signature is not a type as such, we must export the type `Queue` as one of the identifiers; this is done in line 3 with the declaration `type 'a Queue`, which identifies the name `Queue` as an exported type, parameterized by the type variable `'a`. Thus, this signature allows queues to contain elements of arbitrary type (but each queue can only contain elements of a single type). The type `Queue` may be either a type synonym for a previously existing type, or a new type; making it a previously existing type can result in a loss of protection, however (see Exercise 11.29); thus, in our examples we will always use a new type.

```
(1) signature QUEUE =
(2)    sig
(3)    type 'a Queue
(4)    val createq: 'a Queue
(5)    val enqueue: 'a Queue * 'a -> 'a Queue
(6)    val frontq: 'a Queue -> 'a
(7)    val dequeue: 'a Queue -> 'a Queue
(8)    val emptyq: 'a Queue -> bool
(9)    end;
```

Figure 11.15 A QUEUE signature for a queue ADT in ML

Figure 11.16 shows a structure `Queue1` that provides an implementation for the `QUEUE` signature. Using the `QUEUE` signature specification in the first line of this definition (after the colon but before the equal sign) restricts the public interface of this structure to only the features mentioned in the signature, and it requires the structure to implement every feature in the signature in such a way that it has exactly the type specified in the signature. Line 1 of Figure 11.16, thus, defines the name `Queue1` to have type `structure`, implementing the signature `QUEUE`. Lines 2 through 10 then provide the actual `structure` value, which is constructed by the `struct ... end` expression. This expression implements the `Queue` type in exactly the same way as in Figure 11.1, and the internal representation of the `Queue` as a list, including the constructor `Q`, is not accessible outside of the structure because it is not part of the `QUEUE` signature.

```
(1)  structure Queue1: QUEUE =
(2)     struct
(3)     datatype 'a Queue = Q of 'a list
(4)     val createq = Q [];
(5)     fun enqueue(Q lis, elem) = Q (lis @ [elem]);
(6)     fun frontq (Q lis) = hd lis;
(7)     fun dequeue (Q lis) = Q (tl lis);
(8)     fun emptyq (Q []) = true
(9)       | emptyq (Q (h::t)) = false;
(10)    end;
```

Figure 11.16 An ML structure `Queue1` implementing the `QUEUE` signature as an ordinary built-in list with wrapper

The structure `Queue1` can be used in ML as follows:

```
- val q = Queue1.enqueue(Queue1.createq,3);
val q = Q [3] : int Queue1.Queue
- Queue1.frontq q;
val it = 3 : int
- val q1 = Queue1.dequeue q;
val q1 = Q [] : int Queue1.Queue
- Queue1.emptyq q1;
val it = true : bool
```

Note that the internal representation of Queue is not suppressed in the ML interpreter responses as it is with the `abstype` specification of Section 11.2. Also, all names in `Queue1` must be qualified by the name of the structure, unlike the `abstype` specification. As with other languages, this qualification can be removed if desired; in ML the expression that does so is the `open` expression:

```
- open Queue1;
opening Queue1
  datatype 'a Queue = ...
  val createq : 'a Queue
  val enqueue : 'a Queue * 'a -> 'a Queue
  val frontq : 'a Queue -> 'a
  val dequeue : 'a Queue -> 'a Queue
  val emptyq : 'a Queue -> bool
```

The ML interpreter responds to the `open` expression by listing the signature that `Queue1` implements. The features of `Queue1` can now be used without qualification:

```
- val q = enqueue (createq,3);
val q = Q [3] : int Queue
- frontq q;
val it = 3 : int
- val q1 = dequeue q;
val q1 = Q [] : int Queue
- emptyq q1;
val it = true : bool
```

Figure 11.17 shows a second implementation of the `QUEUE` signature, this time using a user-defined linked list instead of the built-in ML list structure. In the definition of the `Queue` data type (lines 3 and 4), we have used two constructors: `Createq` and `Enqueue`. These constructors perform essentially the same operations as the functions `createq` and `enqueue`, so we have given them the same names (except for the uppercase first letter). With these names, the definitions of the functions `frontq`, `dequeue`, and `emptyq` look virtually identical to the equational axiomatic specification for the Queue ADT given in Section 11.1.

```
(1)  structure Queue2: QUEUE =
(2)     struct
(3)     datatype 'a Queue = Createq
(4)                       | Enqueue of 'a Queue * 'a ;
(5)     val createq = Createq;
(6)     fun enqueue(q,elem) = Enqueue (q,elem);
(7)     fun frontq (Enqueue(Createq,elem)) = elem
(8)     | frontq (Enqueue(q,elem)) = frontq q;
(9)     fun dequeue (Enqueue(Createq,elem)) = Createq
```

Figure 11.17 An ML structure `Queue2` implementing the `QUEUE` signature as a user-defined linked list *(continues)*

(continued)

```
(10)                  | dequeue (Enqueue(q,elem))
(11)                          = Enqueue(dequeue q, elem);
(12)    fun emptyq Createq = true | emptyq _ = false;
(13)    end;
```

Figure 11.17 An ML structure `Queue2` implementing the `QUEUE` signature as a user-defined linked list

The `Queue2` structure of Figure 11.17 can be used exactly as the `Queue1` structure of Figure 11.16:

```
- open Queue2;
opening Queue2
  datatype 'a Queue = ...
  val createq : 'a Queue
  val enqueue : 'a Queue * 'a -> 'a Queue
  val frontq : 'a Queue -> 'a
  val dequeue : 'a Queue -> 'a Queue
  val emptyq : 'a Queue -> bool
- val q = enqueue(createq,3);
val q = Enqueue (Createq,3) : int Queue
- frontq q;
val it = 3 : int
- val q1 = dequeue q;
val q1 = Createq : int Queue
- emptyq q1;
val it = true : bool
```

ML signatures and structures satisfy most of the requirements of criteria 1 and 2 for abstract data types from this chapter's introduction. A signature collects a type and its associated operations in one place. The definitions do not depend on any implementation details. A structure collects the implementation details in one place and only exports the information contained in the associated signature. The main difficulty with the ML module mechanism from the point of view of abstract data types is that client code must explicitly state the implementation that is to be used in terms of the module name: Code cannot be written to depend only on the signature, with the actual implementation structure to be supplied externally to the code. Thus, we cannot write:

```
val x = QUEUE.createq;
```

Instead, we must write:

```
val x = Queue1.createq;
```

or

```
val x = Queue2.createq;
```

The main reason for this is that the ML language has no explicit or implicit separate compilation mechanism (or other code aggregation mechanism), so there is no way to externally specify an implementation choice.[8]

11.6 Modules in Earlier Languages

Historically, modules and abstract data type mechanisms began with Simula67 and made significant progress in the 1970s through language design experiments at universities and research centers. Among the languages that have contributed significantly to module mechanisms in Ada, ML, and other languages, are the languages CLU, Euclid, Modula-2, Mesa, and Cedar. To indicate how module mechanisms have developed, we sketch briefly the approaches taken by Euclid, CLU, and Modula-2.

11.6.1 Euclid

In the Euclid programming language, modules are types, so a complex number module is declared as a type:

```
type ComplexNumbers = module
   exports(Complex, add, subtract, multiply,
           divide, negate, makeComplex,
          realPart, imaginaryPart)
   type Complex = record
      var re, im: real
   end Complex

   procedure add (x,y: Complex, var z: Complex) =
   begin
       z.re := x.re + y.re
       z.im := x.im + y.im
   end add

   procedure makeComplex
          (x,y: real, var z:Complex) =
   begin
       z.re := x
       z.im := y
   end makeComplex
            ...
end ComplexNumbers
```

To be able to declare complex numbers, however, we need an actual object of type `ComplexNumbers`—a module type in itself does not exist as an object. Thus, we must declare:

```
var C: ComplexNumbers
```

[8]In fact, most ML systems have a compilation manager with separate compilation that can remove this problem, but such tools are not part of the language itself.

and then we can declare:

```
var z,w: C.Complex
```

and apply the operations of `C`:

```
C.makeComplex(1.0,1.0,z)
C.Add(z,z,w)
```

Note that this implies that there could be two different variables of type `ComplexNumbers` declared and, thus, two different sets of complex numbers:

```
var C1,C2: ComplexNumbers
var x: C1.Complex
var y: C2.Complex


C1.makeComplex(1.0,0.0,x)

C2.makeComplex(0.0,1.0,y)
(* x and y cannot be added together *)
```

When module types are used in a declaration, this creates a variable of the module type, or **instantiates** the module. In Euclid, two different instantiations of `ComplexNumbers` can, therefore, exist simultaneously. This is not true in languages such as Ada or ML, where modules are objects instead of types, with a single instantiation of each. (The Ada generic package can, of course, have several different instantiations, even for the same type; see the discussion in Section 11.4 and Chapter 9.)

11.6.2 CLU

In CLU, modules are defined using the **cluster** mechanism. In the case of complex numbers, the data type `Complex` can be defined directly as a cluster:

```
Complex = cluster is add, multiply,...,
          makeComplex, realPart, imaginaryPart
    rep = struct [re,im: real]
    add = proc (x,y: cvt ) returns (cvt)
      return
      (rep${re: x.re+y.re, im: x.im+y.im})
    end add
      ...
    makeComplex = proc (x,y: real) returns (cvt)
      return (rep${re:x, im:y})
    end makeComplex
```

(continues)

(continued)

```
    realPart = proc(x: cvt) returns (real)
      return(x.re)
    end realPart

end Complex
```

The principal difference of this abstraction mechanism from the previous ones is that the datatype `Complex` is defined directly as a cluster. However, when we define

```
x,y: Complex
```

we do not mean that they should be of the cluster type, but of the representation type within the cluster (given by the **rep** declaration). Thus, a cluster in CLU really refers to two different things: the cluster itself and its internal representation type. Of course, the representation type must be protected from access by the outside world, so the details of the representation type can be accessed only from within the cluster. This is the purpose of the `cvt` declaration (for `convert`), which converts from the external type `Complex` (with no explicit structure) to the internal `rep` type and back again. `cvt` can be used only within the body of a cluster.

Clients can use the functions from `Complex` in a similar manner to other languages:

```
x := Complex$makeComplex(1.0,1.0)
x := Complex$add (x,x)
```

Note the use of the "$" to qualify the operations instead of the dot notation used in Ada and ML.

11.6.3 Modula-2

In Modula-2, the specification and implementation of an abstract data type are separated into a `DEFINITION MODULE` and an `IMPLEMENTATION MODULE`. For the type `Complex`, a Modula-2 specification module would look like this:

```
DEFINITION MODULE ComplexNumbers;

TYPE Complex;

PROCEDURE Add (x,y: Complex): Complex;
PROCEDURE Subtract (x,y: Complex): Complex;
PROCEDURE Multiply (x,y: Complex): Complex;
PROCEDURE Divide (x,y: Complex): Complex;
PROCEDURE Negate (z: Complex): Complex;
PROCEDURE MakeComplex (x,y: REAL): Complex;
PROCEDURE RealPart (z: Complex) : REAL;

PROCEDURE ImaginaryPart (z: Complex) : REAL;

END ComplexNumbers.
```

A Modula-2 `DEFINITION MODULE` contains only definitions or declarations, and only the declarations that appear in the `DEFINITION MODULE` are **exported**, that is, are usable by other modules. The incomplete type specification of type `Complex` in the `DEFINITION MODULE` is called an opaque type; the details of its declaration are hidden in an implementation module and are not usable by other modules. These features are all similar to those of Ada and ML.

A corresponding `IMPLEMENTATION MODULE` in Modula-2 for the `DEFINITION MODULE` is as follows:

```
IMPLEMENTATION MODULE ComplexNumbers;

FROM Storage IMPORT ALLOCATE;

TYPE Complex = POINTER TO ComplexRecord
     ComplexRecord = RECORD
        re, im: REAL;
     END;

PROCEDURE add (x,y: Complex): Complex;
VAR t: Complex;
BEGIN
  NEW(t);
  t^.re := x^.re + y^.re
  t^.im := x^.im + y^.im;
  RETURN t;
END add;

    ...

PROCEDURE makeComplex (x,y: REAL): Complex;
VAR t: Complex;
BEGIN
  NEW(t);
  t^.re := x;
  t^.im := y;
  RETURN t;
END makeComplex;

PROCEDURE realPart (z: Complex) : REAL;
BEGIN
  RETURN z^.re;
END realPart;

PROCEDURE imaginaryPart (z: Complex) : REAL;
BEGIN
  RETURN z^.im;
END imaginaryPart;

END ComplexNumbers.
```

Because of the use of pointers and indirection, functions such as `makeComplex` have to include a call to `NEW` to allocate space for a new complex number, and this in turn requires that `ALLOCATE` be imported from a system `Storage` module. (The compiler converts a call to `NEW` to a call to `ALLOCATE`.)

A client module in Modula-2 uses type `Complex` by **importing** it and its functions from the `ComplexNumbers` module:

```
MODULE ComplexUser;

IMPORT ComplexNumbers;

VAR x,y,z: ComplexNumbers.Complex;

BEGIN
  x := ComplexNumbers.makeComplex(1.0,2.0);
  y := ComplexNumbers.makeComplex(-1.0,1.0);
  z := ComplexNumbers.add(x,y);
        ...
END ComplexUser.
```

Note the qualification in the use of the features from `ComplexNumbers`. An alternative to this is to use a **dereferencing** clause. In Modula-2, the dereferencing clause is the `FROM` clause:

```
MODULE ComplexUser;

FROM ComplexNumbers IMPORT Complex,add,makeComplex:
VAR x,y,z: Complex;
...
BEGIN
  x := makeComplex(1.0,2.0);
  y := makeComplex(-1.0,1.0);
  ...
  z := add(x,y);

END ComplexUser.
```

When a `FROM` clause is used, imported items must be listed by name in the `IMPORT` statement, and no other items (either imported or locally declared) may have the same names as those imported (Modula-2 does not support overloading).

The separation of a module into a `DEFINITION MODULE` and an `IMPLEMENTATION MODULE` in Modula-2 supports the first principle of an abstract data type mechanism: It allows the data type and operation definitions to be collected in one place (the `DEFINITION MODULE`) and associates the operations directly with the type via the `MODULE` name. The use of an opaque type in a `DEFINITION MODULE` supports the second principle of an abstract data type mechanism: the details of the `Complex` data type implementation are separated from its declaration and are hidden in an `IMPLEMENTATION MODULE`

together with the implementation of the operations. Further, a client is prevented from using any of the details in the `IMPLEMENTATION MODULE`.

Since the `DEFINITION MODULE` is independent of any of the details of the `IMPLEMENTATION MODULE`, the implementation can be rewritten without affecting any use of the module by clients, in a similar manner to Ada and C.

11.7 Problems with Abstract Data Type Mechanisms

Abstract data type mechanisms in programming languages use separate compilation facilities to meet protection and implementation independence requirements. The specification part of the ADT mechanism is used as an interface to guarantee consistency of use and implementation. However, ADT mechanisms are used to create types and associate operations to types, while separate compilation facilities are providers of services, which may include variables, constants, or any other programming language entities. Thus, compilation units are in one sense more general than ADT mechanisms. At the same time they are less general, however, in that the use of a compilation unit to define a type does not identify the type with the unit and, thus, is not a true type declaration. Furthermore, units are static entities—that is, they retain their identity only before linking, which can result in allocation and initialization problems. Thus, the use of separate compilation units, such as modules or packages, to implement abstract data types is a compromise in language design. However, it is a useful compromise, because it reduces the implementation question for ADTs to one of consistency checking and linkage.

In this section, we catalog some of the major drawbacks of the ADT mechanisms we have studied (and others we have not). Of course, not all languages suffer from all the drawbacks listed here. Many, but not all, of these drawbacks have been corrected in object-oriented languages. Thus, this section serves as an indication of what to look for in object-oriented languages.

11.7.1 Modules Are Not Types

In C, Ada, and ML, difficulties can arise because a module must export a type as well as operations. It would be helpful instead to define a module to *be a type*. This prevents the need to arrange to protect the implementation details of the type with an ad hoc mechanism such as incomplete or private declarations; the details are all contained in the implementation section of the type.

The fact that ML contains both an `abstype` and a module mechanism emphasizes this distinction. The module mechanism is more general, and it allows the clean separation of specification (signature) from implementation (structure), but a type must be exported. On the other hand, an `abstype` *is* a type, but the implementation of an `abstype` cannot be separated from its specification (although access to the details of the implementation is prevented), and clients of the `abstype` implicitly depend on the implementation.

11.7.2 Modules Are Static Entities

An attractive possibility for implementing an abstract data type is to simply not reveal a type at all, thus avoiding all possibility of clients depending in any way on implementation details, as well as preventing clients from any misuse of a type. For example, in Ada we could write a `Queue` package specification as follows. In the following example, note how the operations must change so that they never return a queue. This is pure imperative programming.

```
generic
  type T is private;
package Queues is
  procedure enqueue(elem:T);
  function frontq return T;
  procedure dequeue;
  function emptyq return Boolean;
end Queues;
```

The actual queue data structure is then buried in the implementation:

```
package body Queues is

  type Queuerep;
  type Queue is access Queuerep;

  type Queuerep is
  record
    data: T;
    next: Queue;
  end record;

  q: Queue;

  procedure enqueue(elem:T) is
    temp: Queue;
  begin
    temp := new Queuerep;
    temp.data := elem;
    if (q /= null) then
       temp.next := q.next;
       q.next := temp;
    else
       temp.next := temp;
    end if;
    q := temp;
  end enqueue;

  function frontq return T is
  begin
    return q.next.data;
  end frontq;
  procedure dequeue is
  begin
```

(continues)

(continued)

```
    if q = q.next then
       q := null;
    else
       q.next := q.next.next;
       q := q.next;
    end if;
  end dequeue;

  function emptyq return Boolean is
  begin
    return q = null;
  end emptyq;

begin
  q := null;
end Queues;
```

Note that this allows us to write initialization code internally within the package body (the last three lines of the above code), which is executed on "elaboration," or allocation time (at the beginning of execution). This eliminates the need for user-supplied code to initialize a queue, and so there is no `createq` procedure.

Normally this would imply that there can only be one queue in a client, since otherwise the entire code must be replicated (try this using a C++ namespace—see Exercise 11.28). This results from the static nature of most module mechanisms (including those of Ada and ML). In Ada, however, the generic package mechanism offers a convenient way to obtain several different queues (even for the same stored type) by using multiple instantiations of the same generic package:

```
with Ada.Text_IO; use Ada.Text_IO;
with Ada.Integer_Text_IO;
use Ada.Integer_Text_IO;
with Queues;

procedure Quser is

package Queue1 is new Queues(Integer);
package Queue2 is new Queues(Integer);

begin
  Queue1.enqueue(3);
  Queue1.enqueue(4);
  Queue2.enqueue(1);
  Queue2.enqueue(2);
  put(Queue1.frontq); -- prints 3
```

(continues)

(continued)

```
    new_line;
    Queue2.dequeue;
    put(Queue2.frontq); -- prints 2
    new_line;
  end Quser;
```

This is still a static alternative, and we could not create a new queue during execution. However, this can still be an extremely useful mechanism.

11.7.3 Modules That Export Types Do Not Adequately Control Operations on Variables of Such Types

In the C and Ada examples of data types `Queue` and `Complex` in Sections 11.3 and 11.4, variables of each type were pointers that had to be allocated and initialized by calling the procedure `createq` or `makeComplex`. However, the exporting module cannot guarantee that this procedure is called before the variables are used; thus, correct allocation and initialization cannot be ensured. Even worse, because queues and, most particularly, complex numbers are pointers in the sample implementations, copies can be made and deallocations performed outside the control of the module, without the user being aware of the consequences, and without the ability to return the deallocated memory to available storage. For instance, in the Ada code

```
z := makeComplex(1.0,0.0);
x := makeComplex(-1.0,0.0);
x := z;
```

the last assignment has the effect of making `z` and `x` point to the same allocated storage and leaves the original value of `x` as garbage in memory. Indeed, the implementation of type `Complex` as a pointer type gives variables of this type pointer semantics, subject to problems of sharing and copying (see Chapter 8).

Even if we were to rewrite the implementation of type `Complex` as a record or struct instead of a pointer (which would wipe out some encapsulation and information hiding in both C and Ada), we would not regain much control. In that case, the declaration of a variable of type `Complex` would automatically perform the allocation, and calling `makeComplex` would perform initialization only. Still, we would have no guarantee that initializations would be properly performed, but at least assignment would have the usual storage semantics, and there would be no unexpected creation of garbage or side effects.

Part of the problem comes from the use of assignment. Indeed, when `x` and `y` are pointer types, `x := y` (`x = y` in C) performs assignment by sharing the object pointed to by `y`, potentially resulting in some unwanted side effects. This is another example of an operation over which a module exporting types has no control. Similarly, the comparison `x = y` tests pointer equality, identity of location in memory, which is not the right test when `x` and `y` are complex numbers:

```
x := makeComplex(1.0,0.0);
y := makeComplex(1.0,0.0);
(* now a test of x = y returns false *)
```

We could write procedures `equal` and `assign` and add them to the operations in the package specification (or the header file in C), but, again, there would be no guarantee that they would be appropriately applied by a client.

In Ada, however, we can use a **limited private type** as a mechanism for controlling the use of equality and assignment. For example:

```
package ComplexNumbers is

type Complex is limited private;

-- operations, including assignment and equality
  ...
function equal(x,y: in Complex) return Boolean;
procedure assign(x: out Complex; y: in Complex);

private
   type ComplexRec;
   type Complex is access ComplexRec;

end ComplexNumbers;
```

Now clients are prevented from using the usual assignment and equality operations, and the package body can ensure that equality is performed appropriately and that assignment deallocates garbage and/or copies values instead of pointers. Indeed, the = (and, implicitly, the inequality / =) operator can be overloaded as described in Chapter 8, so the same infix form can be used after redefinition by the package. Assignment, however, cannot be overloaded in Ada, and there is still no automatic initialization and deallocation.

C++ has an advantage here, since it allows the overloading of *both* assignment (=) and equality (==). Additionally, object-oriented languages solve the initialization problem by the use of **constructors**.

ML also is able to control equality testing by limiting the data type in an `abstype` or `struct` specification to types that do not permit the equality operation. Indeed, ML makes a distinction between types that allow equality testing and types that do not. Type parameters that allow equality testing must be written using a double apostrophe `''a` instead of a single apostrophe `'a`, and a type definition that allows equality must be specified as an `eqtype` rather than just a type:

```
signature QUEUE =
   sig
   eqtype ''a Queue
   val createq: ''a Queue
   ...etc.
  end;
```

Without this, equality cannot be used to test queues. Additionally, assignment in ML is much less of a problem, since standard functional programming practice does away with most uses of that operation (see Chapter 4).

11.7.4 Modules Do Not Always Adequately Represent Their Dependency on Imported Types

Aside from assignment and equality testing, modules often depend on the existence of certain operations on type parameters and may also call functions whose existence is not made explicit in the module specification. A typical example is a data structure such as a binary search tree, priority queue, or ordered list. These data structures all require that the type of the stored values have an order operation such as the less-than arithmetic operation "<" available. Frequently this is not made explicit in a module specification.

For example, C++ templates mask such dependencies in specifications. A simple example is of a template `min` function, which depends on an order operation. In C++ this could be specified as:

```
template <typename T>
T min( T x, T y);
```

Naturally, the implementation of this function shows the dependency on the order operation (but the specification does not):

```
// C++ code
template <typename T>
T min( T x, T y)
// requires an available < operation on T
{ return x < y ? x : y;
}
```

(See Chapter 9 for a discussion on how to make this dependency explicit in C++.)

In Ada it is possible to specify this requirement using additional declarations in the generic part of a package declaration:

```
generic
type Element is private;
with function lessThan (x,y: Element) return Boolean;
package OrderedList is
...
end OrderedList;
```

Now an `OrderedList` package can be instantiated only by providing a suitable `lessThan` function, as in

```
package IntOrderedList is new
   OrderedList (Integer,"<");
```

Such a requirement is called **constrained parameterization**. Without explicit constraints in Ada, no operations are assumed for the type parameter except equality, inequality, and assignment (and, as noted in the previous section, even these are not assumed if the type is declared limited private).

ML also has a feature that allows structures to be explicitly parameterized by other structures; this feature is called a **functor**, because it is essentially a function on structures (albeit one that operates only statically). Given two signatures such as `ORDER` and `ORDERED_LIST` (lines 1–5 and 6–13 of Figure 11.18), a functor that takes an `ORDER` structure as an argument and produces an `ORDERED_LIST` structure as a result can be defined as follows (for a complete example, see lines 14–28 of Figure 11.18):

```
functor OListFUN (structure Order: ORDER):
ORDERED_LIST =
   struct
   ...
   end;
```

Then, given an actual `ORDER` structure value, such as `IntOrder` (lines 29–33), the functor can be applied to create a new `ORDERED_LIST` structure `IntOList`:

```
structure IntOList =
   OlistFUN(structure Order = IntOrder);
```

This makes explicit the appropriate dependencies, but at the cost of requiring an extra structure (`IntOrder`) to be defined that encapsulates the required features.

```
(1)  signature ORDER =
(2)     sig
(3)     type Elem
(4)     val lt: Elem * Elem -> bool
(5)     end;

(6)  signature ORDERED_LIST =
(7)     sig
(8)     type Elem
(9)     type OList
(10)     val create: OList
(11)    val insert: OList * Elem -> OList
(12)    val lookup: OList * Elem -> bool
(13)    end;
(14) functor OListFUN (structure Order: ORDER):
(15) ORDERED_LIST =
(16)    struct
(17)    type Elem = Order.Elem;
(18)    type OList = Order.Elem list;
(19)    val create = [];
(20)    fun insert ([], x) = [x]
(21)      | insert (h::t, x) = if Order.lt(x,h) then x::h::t
(22)                                 else h:: insert (t, x);
(23)    fun lookup ([], x) = false
```

Figure 11.18 The use of a functor in ML to define an ordered list *(continues)*

(continued)

```
(24)      | lookup (h::t, x) =
(25)          if Order.lt(x,h) then false
(26)          else if Order.lt(h,x) then lookup (t,x)
(27)          else true;
(28)    end;

(29) structure IntOrder: ORDER =
(30)    struct
(31)    type Elem = int;
(32)    val lt = (op <);
(33)    end;

(34) structure IntOList =
(35)    OListFUN(structure Order = IntOrder);
```

Figure 11.18 The use of a functor in ML to define an ordered list

With the code of Figure 11.18, the `IntOList` structure can be used as follows:

```
- open IntOList;
opening IntOList
  type Elem = IntOrder.Elem
  type OList = IntOrder.Elem list
  val create : OList
  val insert : OList * Elem -> OList
  val lookup : OList * Elem -> bool
- val ol = insert(create,2);
val ol = [2] : OList
- val ol2 = insert(ol,3);
val ol2 = [2,3] : OList
- lookup (ol2,3);
val it = true : bool
- lookup (ol,3);
val it = false : bool
```

11.7.5 Module Definitions Include No Specification of the Semantics of the Provided Operations

In the algebraic specification of an abstract data type, equations are given that specify the behavior of the operations. Yet in almost all languages no specification of the behavior of the available operations is required. Some experiments have been conducted with systems that require semantic specification and then attempt to determine whether a provided implementation agrees with its specification, but such languages and systems are still unusual and experimental.

One example of a language that does allow the specification of semantics is Eiffel, an object-oriented language. In Eiffel, semantic specifications are given by preconditions, postconditions, and invariants. Preconditions and postconditions establish what must be true before and after the execution of a procedure or function. Invariants establish what must be true about the internal state of the data in an abstract data type. For more on such conditions, see Chapter 13 on formal semantics. For more on Eiffel, see the Notes and References.

A brief example of the way Eiffel establishes semantics is as follows. In a queue ADT as described in Section 11.1, the enqueue operation is defined as follows in Eiffel:

```
enqueue (x:element) is
require
                    not full
ensure
                    if old empty then front = x
                    else front = old front;
                    not empty
end; -- enqueue
```

The `require` section establishes preconditions—in this case, to execute the enqueue operation correctly, the queue must not be full (this is not part of the algebraic specification as given in Section 11.1; see Exercise 11.24). The `ensure` section establishes postconditions, which in the case of the `enqueue` operation are that the newly enqueued element becomes the front of the queue only if the queue was previously empty and that the queue is now not empty. These requirements correspond to the algebraic axioms

frontq(enqueue(q,x)) = if emptyq(q) then x else frontq(q)
emptyq(enqueue(q,x)) = false

of Section 11.1.

In most languages, we are confined to giving indications of the semantic content of operations inside comments.

11.8 The Mathematics of Abstract Data Types

In the algebraic specification of an abstract data type, a data type, a set of operations, and a set of axioms in equational form are specified. Nothing is said, however, about the actual existence of such a type, which is hypothetical until an actual type is constructed that meets all the requirements. In the parlance of programming language theory, an abstract data type is said to have **existential type**—it asserts the existence of an actual type that meets its requirements. Such an actual type is a set with operations of the appropriate form that satisfy the given equations. A set and operations that meet the specification are a **model** for the specification. Given an algebraic specification, it is possible for no model to exist, or many models. Therefore, it is necessary to distinguish an actual type (a model) from a potential type (an algebraic specification). Potential types are called sorts, and potential sets of operations are called

signatures. (Note that this agrees with our previous use of the term signature to denote the syntax of prospective operations.) Thus, a sort is the name of a type, which is not yet associated with any actual set of values. Similarly, a signature is the name and type of an operation or set of operations, which exists only in theory, having no actual operations yet associated with it. A model is then an actualization of a sort and its signature and is called an algebra (since it has operations).

For this reason, algebraic specifications are often written using the sort-signature terminology:

sort queue(element) **imports** boolean

signature:

createq: queue
enqueue: queue × element → queue
dequeue: queue → queue
frontq: queue → element
emptyq: queue → boolean

axioms:

emptyq(createq) = true
emptyq (enqueue (q, x)) = false
frontq(createq) = error
frontq(enqueue(q,x)) = if emptyq(q) then x else frontq(q)
dequeue(createq) = error
dequeue(enqueue(q,x)) = if emptyq(q) then q else enqueue(dequeue(q), x)

Given a sort, a signature, and axioms, we would, of course, like to know that an actual type exists for that specification. In fact, we would like to be able to construct a *unique* algebra for the specification that we would then take to be *the* type represented by the specification.

How does one construct such an algebra? A standard mathematical method is to construct the **free algebra of terms** for a sort and signature and then to form the **quotient algebra** of the equivalence relation generated by the equational axioms. The free algebra of terms consists of all the legal combinations of operations. For example, the free algebra for sort queue(integer) and signature as above has terms such as the following:

createq
enqueue (createq, 2)
enqueue (enqueue(createq, 2), 1)
dequeue (enqueue (createq, 2))
dequeue (enqueue(enqueue (createq, 2), -1))
dequeue (dequeue (enqueue (createq, 3)))
etc.

Note that the axioms for a queue imply that some of these terms are actually equal, for example,

dequeue (enqueue (createq, 2)) = createq

In the free algebra, however, all terms are considered to be different; that is why it is called free. In the free algebra of terms, no axioms are true. To make the axioms true, and so to construct a type that models the specification, we need to use the axioms to reduce the number of distinct elements in the free algebra. The two distinct terms createq and dequeue(enqueue(createq,2)) need to be identified, for example, in order to make the axiom

dequeue(enqueue(q,x)) =
    if emptyq(q) then q else enqueue(dequeue(q),x)

hold. This can be done by constructing an **equivalence relation** == from the axioms: "==" is an equivalence relation if it is symmetric, transitive, and reflexive:

if x == y then y == x (symmetry)
if x == y and y == z then x == z (transitivity)
x == x (reflexivity)

It is a standard task in mathematics to show that, given an equivalence relation == and a free algebra F, there is a unique, well-defined algebra F/== such that $x = y$ in F/== if and only if x == y in F. The algebra F/== is called the quotient algebra of F by ==. Furthermore, given a set of equations, such as the axioms in an algebraic specification, there is a unique "smallest" equivalence relation making the two sides of every equation equivalent and hence equal in the quotient algebra. It is this quotient algebra that is usually taken to be "the" data type defined by an algebraic specification. This algebra has the property that the only terms that are equal are those that are provably equal from the axioms. Thus, in the queue example,

dequeue(enqueue(enqueue(createq,2),3)) =
enqueue (dequeue(enqueue(createq, 2)),3) =
enqueue(createq, 3)

but

enqueue(createq, 2) ≠ enqueue(createq, 3).

For mathematical reasons (coming from category theory and universal algebra), this algebra is called the **initial algebra** represented by the specification, and using this algebra as the data type of the specification results in what are called **initial semantics**.

How do we know whether the algebra we get from this construction really has the properties of the data type that we want? The answer is that we don't, unless we have written the "right" axioms. In general, axiom systems should be **consistent** and **complete**. Consistency says that the axioms should not identify terms that should be different. For example, empty(createq) = true and empty(createq) = false are inconsistent axioms, because they result in false = true; that is, false and true become identified in the initial semantics. Consistency of the axiom system says that the initial algebra is not "too small." Completeness of the axiom system, on the other hand, says that the initial algebra is not "too big"; that is, elements of the initial algebra that should be the same aren't. (Note that adding axioms identifies more elements, thus making the initial algebra "smaller" while taking away axioms makes it "larger.") Another useful but not as critical property for an axiom system is **independence**—that is, no axiom is implied by other axioms. For example, frontq(enqueue(enqueue(createq,x),y)) = x is not independent because it follows from other axioms:

frontq(enqueue(enqueue(createq,x),y)) = frontq(enqueue(createq,x))

since frontq(enqueue(q,y)) = frontq(q) for q = enqueue(createq,x), by the fourth axiom on page 496 applied to the case when $q \neq$ createq, and then

$$\text{frontq(enqueue (createq,}x\text{))} = x$$

by the same axiom applied to the case when q = createq.

Deciding on an appropriate set of axioms is in general a difficult process. In Section 11.1 we gave a simplistic method based on the classification of the operations into constructors and inspectors. Such methods, while useful, are not foolproof and do not cover all cases. Moreover, dealing with errors causes extra difficulties, which we do not study here.

Initial semantics for algebraic specifications are unforgiving because, if we leave out an axiom, we can in general get many values in our data type that should be equal but aren't. More forgiving semantics are given by an approach that assumes that any two data values that cannot be distinguished by inspector operations must be equal. An algebra that expresses such semantics is called (again for mathematical reasons) a **final algebra**, and the associated semantics are called **final semantics**.

A final algebra is also essentially unique, and can be constructed by means similar to the initial algebra construction. To see the difference between these two definitions, we take the example of an integer array abstract data type. An algebraic specification is as follows:

type IntArray **imports** integer

operations:

createIA: IntArray
insertIA: IntArray × integer × integer → IntArray
extractIA: IntArray × integer → integer

variables: A: IntArray; i,j,k: integer;

axioms:

extractIA(createIA,i) = 0
extractIA (insertIA (A, i, j), k) = if $i = k$ then j else extractIA(A, k)

This array data type is essentially an integer array type indexed by the entire set of integers: insertIA(A,i,j) is like A[i] = j, and extractIA(A, i) is like getting the value A[i]. There is a problem, however: In initial semantics, the arrays

$$\text{insertIA (insertIA (createIA,1), 2,2)}$$

and

$$\text{insertIA(insertIA(createIA,2,2),1,1)}$$

are not equal! (There is no rule that allows us to switch the order of inserts.) Final semantics, however, tells us that two arrays must be equal if all their values are equal:

$$A = B \text{ if and only if for all integers } i \text{ extractIA}(A,i) = \text{extractIA}(B,i)$$

This is an example of the **principle of extensionality** in mathematics: Two things are equal precisely when all their components are equal. It is a natural property and is likely to be one we want for abstract

data types, or at least for the IntArray specification. To make the final and initial semantics agree, however, we must add the axiom

insertIA (insertIA(A,i,j),k,m) = insertIA (insertIA(A,k,m),i,j)

In the case of the queue algebraic specification, the final and initial semantics already agree, so we don't have to worry: given two queues p and q,

$p = q$ if and only if p = createq and q = createq
or frontq(p) = frontq(q) and dequeue(p) = dequeue(q)

Exercises

11.1 Discuss how the following languages meet or fail to meet criteria 1 and 2 for an abstract data type mechanism from the beginning of this chapter:
(a) C
(b) Ada
(c) ML
(d) another language of your choice.

11.2 In the algebraic specification of the complex abstract data type in Section 11.1, axioms were given that base complex addition and subtraction on the corresponding real operations. Write similar axioms for **(a)** multiplication and **(b)** division.

11.3 An alternative to writing axioms relating the operations of complex numbers to the corresponding operations of their real and imaginary parts is to write axioms for all the usual properties of complex numbers, such as associativity, commutativity, and distributivity. What problems do you foresee in this approach?

11.4 Finish writing the `abstype Complex` definition of Figure 11.2 in ML (Section 11.2).

11.5 Finish writing the implementation of `package ComplexNumbers` in Section 11.4.

11.6 Write a complex number module in C using the header file approach described in Section 11.3.

11.7 Use your implementation of Exercise 11.4, 11.5, or 11.6 to write a program to compute the roots of a quadratic equation $ax2 + bx + c = 0$. (a, b, and c may be assumed to be either real or complex.)

11.8 Write a new implementation for complex numbers in C, Ada, or ML that uses polar coordinates. Make sure that your interface and client code (e.g., Exercise 11.7) do not change.

11.9 Write an implementation for the queue ADT in C or Ada that uses a doubly linked list with front and rear pointers instead of the singly linked list with rear pointer that was used in this chapter. Make sure that your interface and client code do not change.

11.10 Rewrite the abstype Complex in ML as a signature and structure.

11.11 Implement the Stack ADT as described in Section 11.1 in **(a)** C; **(b)** Ada; **(c)** ML.

11.12 A **double-ended queue**, or **deque**, is a data type that combines the actions of a stack and a queue. Write an algebraic specification for a deque abstract data type assuming the following operations: create, empty, front, rear, addfront, addrear, deletefront, and deleterear.

11.13 Which operations of the complex abstract data type of Section 11.1 are constructors? Which are inspectors? Which are selectors? Which are predicates? Based on the suggestions in Section 11.1, how many axioms should you have?

following operations:

create: SymbolTable
enter: SymbolTable × name × value → SymbolTable
lookup: SymbolTable × name → value
isin: SymbolTable × name → boolean

11.15 Which operations of the SymbolTable abstract data type in Exercise 11.14 are constructors? Which are inspectors? Which are selectors? Which are predicates? Based on the suggestions in Section 11.1, how many axioms should you have?

11.16 Write an algebraic specification for an abstract data type String; think up appropriate operations for such a data type and the axioms they must satisfy. Compare your operations to standard string operations in a language of your choice.

11.17 Write an algebraic specification for an abstract data type bstree (binary search tree) with the following operations:

create: bstree
make: bstree × element 3 bstree → bstree
empty: bstree → boolean
left: bstree → bstree
right: bstree → bstree
data: bstree → element
isin: bstree × element → boolean
insert: bstree × element → bstree

What does one need to know about the element data type in order for a bstree data type to be possible?

11.18 Write C, Ada, or ML implementations for the specifications of Exercises 11.12, 11.14, 11.16, or 11.17. (Use generic packages in Ada as appropriate; use type variables and/or functors in ML as appropriate.)

11.19 Describe the Boolean data type using an algebraic specification.

11.20 Show, using the axioms for a queue from Section 11.1, that the following are true:

(a) frontq(enqueue(enqueue(createq,*x*), *y*)) = *x*

(b) frontq(enqueue(dequeue(enqueue(createq,*x*)), *y*)) = *y*

(c) frontq(dequeue(enqueue(enqueue(createq,*x*), *y*))) = *y*

11.21 A delete operation in a SymbolTable (see Exercise 11.14),

delete: SymbolTable × name → SymbolTable

might have two different semantics, as expressed in the following axioms:

delete(enter(*s*,*x*,*v*),*y*) = if *x* = *y* then *s* else
enter(delete(*s*,*y*),*x*,*v*)

or

delete(enter(*s*,*x*,*v*),*y*) = if *x* = *y* then delete (*s*,*x*) else enter (delete(*s*,*y*),*x*,*v*)

(a) Which of these is more appropriate for a symbol table in a translator for a block-structured language? In what situations might the other be more appropriate?

(b) Rewrite your axioms of Exercise 11.14 to incorporate each of these two axioms.

11.22 Ada requires generic packages to be instantiated with an actual type before the package can be used. By contrast, some other languages (e.g., C++) allow for the use of a type parameter directly in a declaration as follows (in Ada syntax):

```
x: Stack(Integer);
```

Why do you think Ada did not adopt this approach?

11.23 One could complain that the error axioms in the stack and queue algebraic specifications given in this chapter are unnecessarily strict, since for at least some of them, it is possible to define reasonable nonerror values. Give two examples of such axioms. Is it possible to eliminate all error values by finding nonerror substitutes for error values? Discuss.

11.24 In the algebraic specification of a stack or a queue, no mention was made of the possibility that a stack or queue might become full:

$$\text{full: stack} \rightarrow \text{boolean}$$

Such a test requires the inclusion of another operation, namely,

$$\text{size: stack} \rightarrow \text{integer}$$

and a constant

$$\text{maxsize: integer}$$

Write out a set of axioms for such a stack.

11.25 The queue implementations of the text ignored the error axioms in the queue ADT specification (indeed, one of the ML implementations generated warnings as a result). Rewrite the queue implementations in **(a)** Ada or **(b)** ML to use exceptions to implement the error axioms.

11.26 If one attempts to rewrite the Complex abstype of ML as a signature and structure, a problem arises in that infix operators lose their infix status when used by clients, unlike the abstype definitions, which retain their infix status.

(a) Discuss possible reasons for this situation.

(b) Compare this to Ada's handling of infix operators defined in packages.

11.27 Compare the handling of infix operators defined in C++ namespaces to Ada's handling of infix operators defined in packages. Are there any significant differences?

11.28 Write a C++ namespace definition and implementation for a single queue that completely suppresses the queue data type itself. Compare this to the Ada `Queues` package in Section 11.7.2.

11.29 In the ML structure definition `Queue1` in Section 11.5 (Figure 11.16), we noted that, even though the `Queue` data type was protected from misuse by clients, the actual details of the internal representation were still visible in the printed values. Even worse, if we had used the built-in list representation directly (instead of with a `datatype` definition with constructor `Q`), a client could arbitrarily change a `Queue` value without using the interface.

(a) Rewrite the `Queue1` implementation to use a list directly, and show that the internal structure of the data is not protected.

(b) A 1997 addition to the ML module system is the opaque signature declaration, which uses the symbol `:>` instead of just the colon in a signature constraint:

```
structure Queue1:> QUEUE =
   struct
   ...
   end;
```

Show that the use of this opaque signature suppresses all details of the `Queue` type and that an ordinary list can now be used with complete safety.

11.30 The functor `OListFUN` defined in Section 11.7.4 is not safe, in that it exposes the internal details of the `Queue` type to clients, and allows clients full access to these details.

(a) Explain why. (*Hint*: See the previous exercise.)

(b) Could we use an opaque signature declaration to remove this problem, as we did in the previous exercise? Explain.

11.31 An implementation of abstract data type bstree of Exercise 11.17 is unsafe if it exports all the listed operations: A client can destroy the order property by improper use of the operations.

(a) Which operations should be exported and which should not?

(b) Discuss the problems this may create for an implementation and the best way to handle it in C, Ada, or ML.

(c) Write an implementation in C, Ada, or ML based on your answer to part (b).

11.32 Can the integers be used as a model for the Boolean data type? How do the integers differ from the initial algebra for this data type?

11.33 A **canonical form** for elements of an algebra is a way of writing all elements in a standard, or canonical, way. For example, the initial algebra for the queue algebraic specification has the following canonical form for its elements:

$$\text{enqueue(enqueue(. . . enqueue(create,}a1\text{). . .,}an-1\text{),}an\text{)}$$

Show that any element of the initial algebra can be put in this form, and use the canonical form to show that the initial and final semantics of a queue are the same.

11.34 Given the following algebraic specification:

type Item **imports** boolean

operations:

create: Item
push: Item → Item
empty: Item → boolean

variables: *s*: Item;

axioms:

empty(create) = true
empty(push(s)) = false

show that the initial and final semantics of this specification are different. What axiom needs to be added to make them the same?

11.35 Exercise 11.3 noted that the Complex abstract data type may need to have all its algebraic properties, such as commutativity, written out as axioms. This is not necessary if one uses the principle of extensionality (Section 11.8) and assumes the corresponding algebraic properties for the reals. Use the principle of extensionality and the axioms of Exercise 11.2 to show the commutativity of complex addition.

11.36 The principle of extensionality (Section 11.8) can apply to functions as well as data.
(a) Write an extensional definition for the equality of two functions.
(b) Can this definition be implemented in a programming language? Why or why not?
(c) Does the language C, Ada, or ML allow testing of functions for equality? If so, how does this test differ from your extensional definition?

Notes and References

The earliest language to influence the development of abstract data types was Simula67, which introduced abstract data types in the form of classes. The algebraic specification of abstract data types described in Section 11.1 was pioneered by Guttag [1977] and Goguen, Thatcher, and Wagner [1978], who also developed the initial algebra semantics described in Section 11.9. The alternative final algebra semantics described there were developed by Kamin [1983]. A good general reference for abstract data type issues, including mathematical issues discussed in Section 11.8, is Cleaveland [1986]. Namespaces in C++ are discussed in Stroustrup [1997]. Packages in Java are described in Arnold et al. [2000]. More details on ADT specification and implementation in Ada (Section 11.4) can be found in Booch et al. [1993], Barnes [1998], and Cohen [1996]. The ML module mechanisms (signatures, structures, and functors) are described further in Paulson [1996], Ullman [1998], and MacQueen [1988]. CLU examples (Section 11.6) can be found in Liskov [1984]; Euclid examples (Section 11.6) can be found in Lampson et al. [1981]; Modula-2 examples can be found in Wirth [1988a] and King [1988]. Other languages with interesting abstract data type mechanisms are Mesa (Mitchell, Maybury, and Sweet [1979]; Geschke, Morris, and Satterthwaite [1977]) and Cedar (Lampson [1983], and Teitelman [1984]).

For an interesting study of separate compilation and dependencies in ML with references to Ada, Modula-3, and C see Appel and MacQueen [1994].